Build Task — Reusable Chain Module — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python).

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: one that holds the prompt, one that builds the chain, and one that compares the chain to Doc04's raw version and runs the Test Cases.

Every piece is something one exercise above already practiced:

- `ChatPromptTemplate.from_messages(...)` with `EXTRACT_PROMPT` — the Real-world exercise.
- `.with_structured_output(ExtractedInfo)` — the Intermediate and Real-world exercises.
- A function with default parameters that returns the chain — the Basic exercise's `build_chain()`.
- `run_comparison()` and `ComparisonRow` — the Real-world exercise's Approach 2.
- A missing slot raises `KeyError` — the Edge cases exercise.
- `parser.parse(...)` raises `OutputParserException` — the Failure exercise.
- `load_config()` and `get_logger()` — Doc01, copied unchanged.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Copy these in unchanged first: `config.py`, `exceptions.py`, `logging_setup.py` from Doc01's Build Task, and `chat_client.py`, `schemas.py` from Doc04's Build Task.

- **`prompts.py`** — `get_extraction_prompt() -> ChatPromptTemplate`, a function, so `chain.py` and the tests get the same prompt.
- **`chain.py`** — `MODEL_NAME = "gpt-4o-mini"` at the top, and `build_extraction_chain(model_name=MODEL_NAME, temperature=0.0) -> Runnable`. Doc01's `Config` holds only the key and the log level, so the model choice lives here, in one named place.
- **`compare_with_raw_sdk.py`** — the test file: `load_config()` first, then one function per Test Case, each printing one `case N` line.

Difference between Basic and Intermediate: Basic lists where each piece was practiced. Intermediate names the files to copy and the exact function signatures.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
prompts.py:
    get_extraction_prompt(): system EXTRACT_PROMPT + human {text}

chain.py:
    MODEL_NAME = "gpt-4o-mini"
    build_extraction_chain(model_name, temperature):
        log which model, build the prompt and model
        return prompt | model.with_structured_output(ExtractedInfo)

compare_with_raw_sdk.py:
    load config (stop with a clear message if the key is missing)
    case 1: prompt.invoke({}) -> KeyError
    case 2: parser.parse("plain text") -> OutputParserException
    case 3: run_comparison(3 inputs) -> count names that match
```

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# chain.py
MODEL_NAME = "gpt-4o-mini"


def build_extraction_chain(
    model_name: str = MODEL_NAME, temperature: float = 0.0
) -> Runnable:
    logger = get_logger(__name__)
    logger.info(f"Building the extraction chain with {model_name}")
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=model_name, temperature=temperature)
    return prompt | model.with_structured_output(ExtractedInfo)
```

Add the imports, write `prompts.py`, then copy `run_comparison()` from the Real-world exercise into `compare_with_raw_sdk.py` — change only where the chain comes from. Add the two other test functions, then compare against the Solution.

Difference between Basic and Intermediate: Basic is the plan. Intermediate is `chain.py` almost complete.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Copied in unchanged: `config.py`, `exceptions.py`, `logging_setup.py` (Doc01) and `chat_client.py`, `schemas.py` (Doc04). Run everything from inside `practice/build_task/`.

Basic Version

Approach 1 — a fixed chain and a plain loop

`prompts.py` is the same as in the Intermediate version below.

```
# chain.py
from langchain_openai import ChatOpenAI

from prompts import get_extraction_prompt
from schemas import ExtractedInfo


def build_extraction_chain():
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return prompt | model.with_structured_output(ExtractedInfo)
```

```
# compare_with_raw_sdk.py
from chain import build_extraction_chain
from chat_client import create_client, get_structured_reply
from config import load_config

load_config()
client = create_client()
chain = build_extraction_chain()

test_inputs = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]
for text in test_inputs:
    raw_result = get_structured_reply(client, text)
```

```
    lcel_result = chain.invoke({"text": text})
    print(text, "-> same name:", raw_result.name == lcel_result.name)
```

Expected output:

```
Maria wants to reset her password -> same name: True
Hi, I'm Tom and my invoice looks wrong -> same name: True
Ahmed here, please cancel my order -> same name: True
```

This proves the two versions agree. But the model name is buried inside the function, nothing is logged, a missing key crashes with a traceback, and Test Cases 2 and 3 (missing slot, wrong shape) aren't checked at all.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — one prompt, one chain builder, one test file

Story — `prompts.py`: the wording is a separate concern from how the chain is built. It reuses Doc04's `EXTRACT_PROMPT`, so the LCEL and raw versions get the same words. **If not:** the two versions would get different prompts, and a mismatch would prove nothing.

```
# prompts.py
from langchain_core.prompts import ChatPromptTemplate

from chat_client import EXTRACT_PROMPT


def get_extraction_prompt() -> ChatPromptTemplate:
    # why: the same system rules Doc04's raw call sends, word for word —
    # the comparison is only fair if both versions get the same prompt
    return ChatPromptTemplate.from_messages(
        [("system", EXTRACT_PROMPT), ("human", "{text}")]
    )
```

Story — `chain.py`: the one module later documents import instead of writing raw SDK calls again. The model name is a named constant with a default parameter, the same shape as the Basic exercise's `build_chain()`. **If not:** every later file would build its own `ChatOpenAI(...)`, and a model swap would mean a search through all of them.

```
# chain.py
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from logging_setup import get_logger
from prompts import get_extraction_prompt
from schemas import ExtractedInfo

# why: the model choice lives in one named place — a model swap is a
# one-line change here, not a search through every file
MODEL_NAME = "gpt-4o-mini"


def build_extraction_chain(
    model_name: str = MODEL_NAME, temperature: float = 0.0
```

```
) -> Runnable:
    # how: the same default-parameter shape as the Basic exercise's
    # build_chain() — callers can pass nothing, or pick another model
    logger = get_logger(__name__)
    logger.info(f"Building the extraction chain with {model_name}")

    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=model_name, temperature=temperature)
    return prompt | model.with_structured_output(ExtractedInfo)
```

Story — `compare_with_raw_sdk.py` : the Build Task's test file — one function per Test Case, in the plain `case N` print style of Doc04's `test_chat_client.py`. `run_comparison()` is the Real-world exercise's, unchanged except for where the chain comes from. **If not:** a prompt change could quietly make the two versions disagree, and nothing would tell you.

```
# compare_with_raw_sdk.py
# Runs the 3 Test Cases from the README. Run it from inside this folder:
# python compare_with_raw_sdk.py
# Case 3 makes real API calls — needs a valid OPENAI_API_KEY in .env.
import sys
from dataclasses import dataclass

from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import PydanticOutputParser

from chain import build_extraction_chain
from chat_client import create_client, get_structured_reply
from config import load_config
from exceptions import MissingConfigError
from prompts import get_extraction_prompt
from schemas import ExtractedInfo

# how: every input has a name, so neither version returns None here
TEST_INPUTS = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]


@dataclass
class ComparisonRow:
    input_text: str
    raw_result: ExtractedInfo
    lcel_result: ExtractedInfo
    matched: bool
```

```
def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
    # why: copied from the Real-world exercise's Approach 2 — only the
    # chain now comes from chain.py
    client = create_client()
    lcel_chain = build_extraction_chain()
    rows = []
    for text in test_inputs:
        raw_result = get_structured_reply(client, text)
        lcel_result = lcel_chain.invoke({"text": text})
        # why: a name has one right answer; intent wording can differ
        matched = raw_result.name == lcel_result.name
        rows.append(ComparisonRow(text, raw_result, lcel_result, matched))
    return rows


def test_missing_variable() -> None:
    # case 1: a missing {text} fails while building the prompt (Edge cases)
    prompt = get_extraction_prompt()
    try:
        prompt.invoke({})
        print("case 1 (missing text)  -> no error (wrong!)")
    except KeyError:
        print("case 1 (missing text)  -> KeyError")


def test_bad_shape() -> None:
    # case 2: a reply that isn't the shape raises a named error (Failure)
    parser = PydanticOutputParser(pydantic_object=ExtractedInfo)
    try:
        parser.parse("Maria wants a new password")
        print("case 2 (wrong shape)   -> accepted (wrong!)")
    except OutputParserException:
        print("case 2 (wrong shape)   -> OutputParserException")


def test_chain_matches_raw() -> None:
    # case 3: the LCEL chain and Doc04's raw call agree on every name
    rows = run_comparison(TEST_INPUTS)
    matched = 0
    for row in rows:
        if row.matched:
            matched = matched + 1
    print(f"case 3 (chain vs raw)  -> {matched}/{len(rows)} names match")
    for row in rows:
        if not row.matched:
```

```
            print(f"  MISMATCH on: {row.input_text}")
            print(f"    raw:  {row.raw_result}")
            print(f"    lcel: {row.lcel_result}")


def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops here, clearly
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    test_missing_variable()
    test_bad_shape()
    test_chain_matches_raw()


if __name__ == "__main__":
    main()
```

Expected output (case 3 makes real calls):

```
case 1 (missing text)  -> KeyError
case 2 (wrong shape)   -> OutputParserException
case 3 (chain vs raw)  -> 3/3 names match
```

Cases 1 and 2 need no API call — they run for free, every time. If a name ever differs in case 3, the `MISMATCH` lines show both results.

Difference from Basic: the model name is a named constant with a default parameter, the chain logs which model it uses, a missing key stops with one clear line, and all 3 Test Cases are checked — the two free ones on every run.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Approach 2 — the key from an optional `Config`, so a test needs no `.env`

Story — `build_extraction_chain(config: Config | None = None)`: the Basic exercise's Approach 2, applied here. `None` means "load the real settings with `load_config()`"; a test passes its own `Config`, and the key goes to `ChatOpenAI` from the one place that checked it. **If not:** building the chain in a test would always need a real key in `.env`.

```
# chain.py
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from config import Config, load_config
from logging_setup import get_logger
from prompts import get_extraction_prompt
from schemas import ExtractedInfo

MODEL_NAME = "gpt-4o-mini"


def build_extraction_chain(
    config: Config | None = None, model_name: str = MODEL_NAME
) -> Runnable:
    # why: the Basic exercise's build_chain() shape (Approach 2) — None
    # means "load the real settings"; a test can pass its own Config
    if config is None:
        config = load_config()
    logger = get_logger(__name__)
    logger.info(f"Building the extraction chain with {model_name}")

    prompt = get_extraction_prompt()
    # how: the key comes from Doc01's checked Config
    model = ChatOpenAI(
        model=model_name, temperature=0, api_key=config.openai_api_key
    )
    return prompt | model.with_structured_output(ExtractedInfo)
```

`compare_with_raw_sdk.py` doesn't change — it calls `build_extraction_chain()` with nothing, so the real settings are loaded. A test can build the chain with a made-up `Config`:

```
# check_config.py
from chain import build_extraction_chain
```

```
from config import Config

test_config = Config(openai_api_key="sk-test-not-real", log_level="INFO")
chain = build_extraction_chain(test_config)
print(f"built with a test Config: {type(chain).__name__}")
```

Expected output (no `.env` needed):

```
built with a test Config: RunnableSequence
```

Difference from Approach 1: the chain takes its key from a `Config` — loaded by default, or passed in — instead of `ChatOpenAI` reading `OPENAI_API_KEY` on its own.

Which one should you actually use?

The Intermediate version is the whole Build Task: a chain module wired to one named model setting, and a test file that proves all 3 Test Cases. Approach 2 is worth it once you test the chain without a real key. Turning `compare_with_raw_sdk.py` into `pytest` tests is Doc13's topic — not needed here.